

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Thursday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 5.4(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 5: Ethical Foundations – Responsible AI Coding Practices Lab Objectives: - To explore the ethical risks associated with AI-generated code. - To recognize issues related to security, bias, transparency, and copyright. - To reflect on the responsibilities of developers when using AI tools in software development. - To promote awareness of best practices for responsible and ethical AI coding.	Week3 - Thursday	

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices..

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

Expected Output #1:

- A script with inline Copilot-suggested code and comments explaining how to safeguard or anonymize user information (e.g., hashing emails, not storing data unencrypted).

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis. Then prompt Copilot to identify and handle potential biases in the data.

Expected Output #2:

- Copilot-generated code with additions or comments addressing bias mitigation strategies (e.g., balancing dataset, removing offensive terms).

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.

Expected Output #3:

- Copilot suggestions that include explanations, fairness checks (e.g., avoiding favoritism), and user feedback options in the code.

Task Description #4:

- Prompt Copilot to generate logging functionality in a Python web application. Then, ask it to ensure the logs do not record sensitive information.

Expected Output #4:

- Logging code that avoids saving personal identifiers (e.g., passwords, emails), and includes comments about ethical logging practices.

Task Description #5:

- Ask Copilot to generate a machine learning model. Then, prompt it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

Expected Output #5:

- Copilot-generated model code with a README or inline documentation suggesting responsible usage, limitations, and fairness considerations.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
How to anonymize or protect this data	0.5

	Identify and handle potential biases in the data.	0.5		
	Follow ethical guidelines like transparency and fairness.	0.5		
	logs do not record sensitive information.	0.5		
	How to use the model responsibly	0.5		
	Total	2.5 Marks		

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

Expected Output #1:

- A script with inline Copilot-suggested code and comments explaining how to safeguard or anonymize user information (e.g., hashing emails, not storing data unencrypted).

PROMPT:

generate a Python script that collects user data (e.g., name, age, email). Then add comments on how to anonymize or protect this data.

```

5.4.1.py
1  # Python script to collect user data
2
3  def collect_user_data():
4      user_data = {}
5      user_data['name'] = input("Enter your name: ")
6      user_data['age'] = input("Enter your age: ")
7      user_data['email'] = input("Enter your email: ")
8      return user_data
9
10 def main():
11     data = collect_user_data()
12     print("Collected Data:", data)
13
14     # --- Data Protection & Anonymization Comments ---
15     # 1. Avoid storing raw personal data unless necessary.
16     # 2. To anonymize, remove or hash identifiers (e.g., use hash(data['email'])).
17     # 3. Store data in encrypted files or databases.
18     # 4. Limit access to sensitive data.
19     # 5. Regularly delete data that is no longer needed.
20     # 6. Use pseudonyms or user IDs instead of real names/emails in logs or analytics.
21
22 if __name__ == "__main__":
23     main()
24
Enter your name: Akshitha
Enter your age: 19
Enter your age: 19
Enter your email: akshithareddy1621@gmail.com
Collected Data: {'name': 'Akshitha', 'age': '19', 'email': 'akshithareddy1621@gmail.com'}
PS C:\Users\Ashwitha\Desktop\AIAC\5.4>

```

PROMPT2

Generate a Python script that collects user data (e.g., name, age, email). Ensure that the email is stored

in a protected form, such as masking it with stars (e.g., j***@gmail.com). Add comments in the code explaining how to anonymize or protect personal data (like removing identifiers, masking, or encrypting sensitive fields)."

```
1  # Python script to collect user data with email masking for privacy
2
3  def mask_email(email):
4      """
5      Masks the email address for privacy.
6      Example: john.doe@gmail.com -> j***@gmail.com
7      """
8      try:
9          local, domain = email.split('@')
10         if len(local) > 1:
11             masked_local = local[0] + '*' * (len(local) - 1)
12         else:
13             masked_local = '*'
14         return masked_local + '@' + domain
15     except Exception:
16         # If email format is invalid, return as is
17         return email
18
19 def collect_user_data():
20     user_data = {}
21     user_data['name'] = input("Enter your name: ")
22     user_data['age'] = input("Enter your age: ")
23     email = input("Enter your email: ")
24     user_data['email'] = mask_email(email) # Store masked email for protection
25     return user_data
26
27 def main():
28     data = collect_user_data()
29     print("Collected Data (with protected email):", data)
30
31     # --- Data Protection & Anonymization Comments ---
32     # 1. Avoid storing raw personal data unless absolutely necessary.
33     # 2. Mask or anonymize sensitive fields (e.g., use mask_email for emails).
34     # 3. To further anonymize, remove or hash identifiers (e.g., hash(data['email'])).
35     # 4. For strong protection, encrypt sensitive data before storage.
36     # 5. Limit access to personal data and regularly delete unnecessary records.
37     # 6. Use pseudonyms or user IDs instead of real names/emails in logs or analytics.
38
39 if __name__ == "__main__":
40     main()
41
```

```
Enter your name: Akshitha Reddy
Enter your age: 19
Enter your age: 19
Enter your email: akshithareddy1621@gmail.com
Collected Data (with protected email): {'name': 'Akshitha Reddy', 'age': '19', 'email': 'a*****@
Collected Data (with protected email): {'name': 'Akshitha Reddy', 'age': '19', 'email': 'a*****@
gmail.com'}
PS C:\Users\Ashwitha\Desktop\AIAC\5.4>
```

Task Description #2:

- Ask Copilot generate a Python function for sentiment analysis. Then prompt Copilot to identify and handle potential biases in the data.

Expected Output #2:

- Copilot-generated code with additions or comments addressing bias mitigation strategies (e.g., balancing dataset, removing offensive terms)

PROMPT:

generate a Python code for sentiment analysis. Then identify and handle potential biases in the data ,provide an option for input

```
1  import re
2  from collections import Counter
3
4  def preprocess_text(text):
5      # Lowercase and remove non-alphabetic characters
6      text = text.lower()
7      text = re.sub(r'^a-z\s', '', text)
8      return text
9
10 def simple_sentiment_analysis(text, positive_words, negative_words):
11     words = preprocess_text(text).split()
12     pos_count = sum(1 for word in words if word in positive_words)
13     neg_count = sum(1 for word in words if word in negative_words)
14     if pos_count > neg_count:
15         return "Positive"
16     elif neg_count > pos_count:
17         return "Negative"
18     else:
19         return "Neutral"
20
21 def detect_and_handle_bias(text, bias_words):
22     words = preprocess_text(text).split()
23     bias_found = [word for word in words if word in bias_words]
24     if bias_found:
25         print("Warning: Potential bias detected in the input text:", ', '.join(set(bias_found)))
26         # Optionally, remove bias words for fairer analysis
27         words = [word for word in words if word not in bias_words]
28         return ' '.join(words)
29     return ' '.join(words)
30
```

```

31  ():
32  # Simple word lists (in practice, use more comprehensive lists)
33  positive_words = {"good", "happy", "excellent", "fortunate", "correct", "superior", "love", "great", "positive"}
34  negative_words = {"bad", "sad", "poor", "unfortunate", "wrong", "inferior", "hate", "terrible", "negative"}
35  # Simple bias words (could be expanded based on domain)
36  bias_words = {"always", "never", "everyone", "nobody", "obviously", "clearly"}
37
38  print("Sentiment Analysis Tool (with Bias Detection)")
39  user_input = input("Enter text for sentiment analysis: ")
40
41  # Detect and handle bias
42  processed_text = detect_and_handle_bias(user_input, bias_words)
43
44  # Perform sentiment analysis
45  sentiment = simple_sentiment_analysis(processed_text, positive_words, negative_words)
46  print("Sentiment:", sentiment)
47
48  print("\nEthical Notice:")
49  print("1. This tool uses simple word lists and may not capture all nuances of sentiment or bias.")
50  print("2. Bias detection is based on a small set of potentially loaded words; real-world bias is more complex.")
51  print("3. For critical applications, use advanced models and regularly audit for fairness and bias.")
52
53  if __name__ == "__main__":
54      ()
55

```

```

● Enter text for sentiment analysis: i got good CGPA in last sem
Sentiment: Positive

```

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.

Expected Output #3:

- Copilot suggestions that include explanations, fairness checks (e.g., avoiding favoritism), and user feedback options in the code.

PROMPT:

write a Python program that recommends products based on user history collected from the user. Ask it to follow ethical guidelines like transparency and fairness

```

5.4.3.py
1  # Python program to recommend products based on user history
2  # Ethical Guidelines: This program is transparent about how recommendations are made and aims to treat
3
4  def collect_user_history():
5      """
6      Collects product history from the user.
7      Returns a list of products the user has interacted with.
8      """
9      print("We collect your product history to recommend items you may like.")
10     print("Please enter the names of products you have purchased or viewed, separated by commas.")
11     history_input = input("Product history: ")
12     user_history = [item.strip().lower() for item in history_input.split(",") if item.strip()]
13     return user_history
14
15 def recommend_products(user_history, product_catalog):
16     """
17     Recommends products based on user history.
18     The recommendation is based on simple matching of categories.
19     """
20     # Transparency: Recommendations are based on matching categories of products you've interacted with
21     # Fairness: All products in the catalog are considered equally, and no personal attributes are used
22     recommended = set()
23     user_categories = set()
24     for product in user_history:
25         if product in product_catalog:
26             user_categories.add(product_catalog[product]['category'])
27     for product, info in product_catalog.items():
28         if info['category'] in user_categories and product not in user_history:
29             recommended.add(product)
30     return list(recommended)
31
32 def main():
33     # Example product catalog
34     product_catalog = {

```

```

35     'laptop': {'category': 'electronics'},
36     'headphones': {'category': 'electronics'},
37     'novel': {'category': 'books'},
38     'cookbook': {'category': 'books'},
39     't-shirt': {'category': 'clothing'},
40     'jeans': {'category': 'clothing'},
41     'smartphone': {'category': 'electronics'},
42     'jacket': {'category': 'clothing'},
43 }
44
45 print("Welcome to the Product Recommender!")
46 print("We value your privacy and fairness. Recommendations are made only from your provided product history and are not influenced by personal attributes.")
47 user_history = collect_user_history()
48 recommendations = recommend_products(user_history, product_catalog)
49
50 print("\nBased on your history, we recommend the following products:")
51 if recommendations:
52     for product in recommendations:
53         print("-", product.title())
54 else:
55     print("No recommendations available based on your history.")
56
57 print("\nEthical Notice:")
58 print("1. Your product history is used only for generating recommendations during this session.")
59 print("2. No personal or sensitive data is collected or stored.")
60 print("3. Recommendations are based solely on product categories, ensuring fairness and transparency.")
61
62 if __name__ == "__main__":
63     main()
64
65 Ctrl+L to chat, Ctrl+K to generate

```

```

Welcome to the Product Recommender!
We value your privacy and fairness. Recommendations are made only from your provided product history and are not influenced by personal attributes.
We collect your product history to recommend items you may like.
Please enter the names of products you have purchased or viewed, separated by commas.
Product history: laptop

Please enter the names of products you have purchased or viewed, separated by commas.
Product history: laptop

Product history: laptop

```